WEEK 4 · LESSON 21 OF 24 · HANDOUT

Generative AI Beyond Text: Images, Audio and Diffusion

Study time: **about 60 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=iv-5mZ_9CPY — open the lesson online to watch inside the course.

🎓 **Go deeper:** 3Blue1Brown: How AI connects text and images — https://www.youtube.com/watch?v=j5h9dBtWZXM

🎓 **Go deeper:** EuroPython 2023: Understanding Neural Network Architectures with Attention and Diffusion — https://www.youtube.com/watch?v=Clh0nJRMvNs

🎓 **Go deeper:** EuroPython 2023: Whisper AI — Live English Subtitles for 96 Languages — https://www.youtube.com/watch?v=WpKzFqrQXdw

Your goal in this lesson: Explain how diffusion models generate images, train a tiny diffusion model yourself, send an image to a multimodal LLM, and recognise the main tools and risks of generated media.

The generative AI family

Task	Main technique	Examples of systems
Text → text	Transformer language models	Claude, Gemini, GPT, Llama
Text → image	**Diffusion** models guided by text	Stable Diffusion, FLUX, Imagen
Image → text	Multimodal LLMs ("vision")	Claude, Gemini, GPT can read images
Speech → text	Transformer speech models	Whisper (open, many languages)
Text → speech, music, video	Transformers and diffusion	Many commercial and open systems

Text and images are linked by models such as **CLIP**, trained so that an image and its caption get **similar embeddings** (Lesson 13). That shared embedding space is how a text prompt can guide image generation.

How diffusion works

1. **Forward process (easy):** take a real image and add a little random noise, many times, until it is pure noise.
2. **Train a network** to look at a noisy image and the noise level, and predict the noise that was added.
3. **Generate (reverse process):** start from pure noise and repeatedly remove the predicted noise. A clean, new image appears.

Image models do this with millions of pixels and a text prompt as extra input. We will do it with 2-D points so it trains in about a minute on a CPU — the maths is the same.

The data: points shaped like two moons

```
import torch
import torch.nn as nn
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons

torch.manual_seed(0)
data = torch.tensor(make_moons(4000, noise=0.05, random_state=0)[0], dtype=torch.float32)
data = (data - data.mean(0)) / data.std(0)

T = 100
betas = torch.linspace(1e-4, 0.1, T)                # how much noise is added at each step
alphas = 1 - betas
alpha_bar = torch.cumprod(alphas, dim=0)            # how much of the original survives after
t steps

def add_noise(x0, t):
    noise = torch.randn_like(x0)
    return alpha_bar[t].sqrt()[:, None] * x0 + (1 - alpha_bar[t]).sqrt()[:, None] * noise,
noise

print(f"signal left after 10 steps: {alpha_bar[9].sqrt():.2f}, after 100 steps:
{alpha_bar[-1].sqrt():.3f}")
```

After 100 steps almost nothing of the original is left — just noise.

Train the denoiser

```
denoiser = nn.Sequential(nn.Linear(3, 128), nn.SiLU(), nn.Linear(128, 128), nn.SiLU(),
                         nn.Linear(128, 128), nn.SiLU(), nn.Linear(128, 2))
opt = torch.optim.Adam(denoiser.parameters(), lr=2e-3)

for step in range(3000):
    x0 = data[torch.randint(0, len(data), (512,))]
    t = torch.randint(0, T, (512,))
    xt, noise = add_noise(x0, t)
    pred = denoiser(torch.cat([xt, (t / T)[:, None]], dim=1))   # input: noisy point + noise
level
    loss = ((pred - noise) ** 2).mean()                          # learn to predict the noise
    opt.zero_grad(); loss.backward(); opt.step()
    if step % 1000 == 0 or step == 2999:
        print(f"step {step:4d}  loss {loss.item():.3f}")
```

Generate new points from pure noise

```
@torch.no_grad()
def sample(n=2000):
    x = torch.randn(n, 2)                                          # start: pure noise
    for t in reversed(range(T)):
        eps = denoiser(torch.cat([x, torch.full((n, 1), t / T)], dim=1))
        x = (x - betas[t] / (1 - alpha_bar[t]).sqrt() * eps) / alphas[t].sqrt()
        if t > 0:
            x = x + betas[t].sqrt() * torch.randn_like(x)          # a little fresh noise
    return x

generated = sample()
noise_only = torch.randn(2000, 2)
dist = lambda pts: torch.cdist(pts, data).min(dim=1).values.median().item()
print(f"median distance to the real data — generated: {dist(generated):.3f}, pure noise:
{dist(noise_only):.3f}")

fig, axes = plt.subplots(1, 3, figsize=(12, 4))
for ax, pts, title in zip(axes, [data[:2000], noise_only, generated], ["real data", "start:
noise", "generated"]):
    ax.scatter(pts[:, 0], pts[:, 1], s=2); ax.set_title(title); ax.set_xlim(-3, 3);
ax.set_ylim(-3, 3)
plt.show()
```

The generated points form the two moons again, far closer to the real data than random noise — **new** points, not copies. An image diffusion model does exactly this, with a large network (often a transformer) predicting the noise in every pixel, guided by the text prompt's embedding.

Images as input: multimodal LLMs

Modern LLMs can also **read** images — receipts, charts, screenshots, handwritten forms. You send the image as base64 data alongside your question:

```
import importlib.util, os, subprocess, sys

if importlib.util.find_spec("anthropic") is None:          # install once per Colab session
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", "anthropic"], check=True)

try:
    from google.colab import userdata                     # Colab: key saved in the Secrets
panel (key icon)
    os.environ["ANTHROPIC_API_KEY"] = userdata.get("ANTHROPIC_API_KEY")
except ImportError:
    pass                                                    # not in Colab: set
ANTHROPIC_API_KEY in your environment
except Exception:
    print("Add a secret called ANTHROPIC_API_KEY in Colab's Secrets panel, then run this cell
again.")

import anthropic

client = anthropic.Anthropic()        # reads ANTHROPIC_API_KEY from the environment
MODEL = "claude-haiku-5-5"            # fast and low-cost; check docs.claude.com for current
model names

def ask(prompt, system=None, max_tokens=500):
    extra = {"system": system} if system else {}
    response = client.messages.create(model=MODEL, max_tokens=max_tokens,
                                      messages=[{"role": "user", "content": prompt}], **extra)
    return response.content[0].text
```

```
import base64
import io

fig, ax = plt.subplots(figsize=(4, 3))
ax.bar(["Mon", "Tue", "Wed", "Thu", "Fri"], [12, 18, 9, 22, 30], color="teal")
ax.set_title("Mobile money payments per day")
buf = io.BytesIO(); fig.savefig(buf, format="png"); plt.close(fig)
image_b64 = base64.standard_b64encode(buf.getvalue()).decode()

response = client.messages.create(
    model=MODEL, max_tokens=300,
    messages=[{"role": "user", "content": [
        {"type": "image", "source": {"type": "base64", "media_type": "image/png", "data":
image_b64}},
        {"type": "text", "text": "Which day had the most payments? Answer in one sentence."},
    ]}],
)
print(response.content[0].text)
```

Images cost tokens too (roughly in proportion to their size), so resize big photos before sending.

Speech

Open speech-recognition models such as **Whisper** transcribe dozens of languages; quality differs a lot between languages, so test with real recordings of your users — including Kinyarwanda and code-switching between Kinyarwanda, English and French.

Risks of generated media

- **Deepfakes:** fake images, voices or videos of real people used for fraud, harassment or political lies. Never generate media of a real person without their consent.

- **Copyright and style:** models are trained on others' work; follow the licence and do not imitate living artists' work or brands.
- **Labelling:** say clearly when an image, voice or text is AI-generated. Some systems add invisible watermarks.
- **Bias:** generated people often follow stereotypes from the training data — check outputs for who is shown and how.

Key terms

- **Diffusion model** — learns to remove noise step by step, generating data from pure noise.
- **Forward / reverse process** — adding noise / removing noise.
- **Noise schedule** — how much noise is added at each step (betas).
- **CLIP** — a model that puts images and text in the same embedding space.
- **Multimodal model** — a model that accepts several kinds of input, such as text and images.
- **Deepfake** — realistic fake media of a real person.

Remember this: Diffusion models generate by learning to remove noise; multimodal LLMs read images and other media; generated media must be used with consent, labels and care.

Name: _____ Date: _____

Activity

Task 1

Train the denoiser for only 300 steps. What do the generated points look like?

Task 2

Change `T` to 20. Is generation faster? Is it worse?

Task 3

Replace the moons with `make_circles` from scikit-learn and train again.

Task 4

Send a photo of a printed receipt (no personal data) to the model and ask it to extract the total as JSON.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. How does a diffusion model generate an image?
   - A. It draws lines
   - B. It starts from pure noise and removes predicted noise step by step
   - C. It copies a training image
   - D. It searches the internet
2. What does the denoiser network learn to predict?
   - A. The noise that was added
   - B. The colour palette
   - C. The image label
   - D. The next word
3. What does CLIP do?
   - A. Puts images and their captions in a shared embedding space
   - B. Compresses videos
   - C. Translates text
   - D. Removes noise
4. How do you send an image to a multimodal LLM through the API?
   - A. As base64 data in an image content block with your question
   - B. Only as a URL in the system prompt
   - C. By printing it
   - D. You cannot
5. Which use of generated media is unacceptable?
   - A. A chart
   - B. A labelled illustration for a lesson
   - C. A logo you own
   - D. A fake voice of a real person without consent
6. Why test speech models on Kinyarwanda recordings?
   - A. Only for speed
   - B. Quality differs a lot between languages
   - C. They are always perfect
   - D. Kinyarwanda is not allowed

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.